Bosch Global Software Technologies

PDF Document Extraction Pipeline

Fully Local, End-to-End Structured Data Extraction from PDF Documents

No proprietary APIs · No cloud dependency · Runs entirely on your machine

Author	Aman Pant (MTQ3KOR)
Mentors	Ganesh Jayabal, Ishan Dindorkar
Manager	Arun Govinda Rao
Organization	Bosch Global Software Technologies
Department	ERD – Engineering Research & Development, Bengaluru
Role	Intern 2026
Report Date	April 2026

Acknowledgement

I would like to express my sincere gratitude to **Bosch Global Software Technologies** for providing me with the opportunity to work on this project as part of the **ERD Internship 2026** program.

I am deeply thankful to my mentors, **Ganesh Jayabal** and **Ishan Dindorkar**, for their continuous guidance, technical feedback, and support throughout the development of this work. Their insights helped shape the direction of the pipeline and improved the overall quality of the implementation.

I would also like to thank **Arun Govinda Rao** for his supervision, encouragement, and valuable inputs during the course of this internship.

I gratefully acknowledge the contributions of the open-source communities behind **GLM-OCR**, **PP-DocLayout-V3**, **Qwen2.5**, **Donut**, **LLMware**, **Tab2DB**, **Granite Vision**, **Ollama**, and **vLLM**. These tools and frameworks played an important role in enabling the development of a fully local, privacy-preserving document extraction pipeline.

Aman Pant (MTQ3KOR)
ERD Intern 2026
Bosch Global Software Technologies

Contents

1 Abstract

This report documents the design, implementation, and evaluation of a fully local, end-to-end pipeline for extracting structured JSON data from commercial PDF documents such as purchase orders and shipping bills.

The pipeline operates in two stages. Stage 1 performs OCR and raw extraction using `GLM-OCR` (0.9B parameters) with `PP-DocLayout-V3` for layout analysis, producing Markdown and structured JSON outputs. Stage 2 applies a hybrid AI and rule-based extraction engine that maps OCR outputs to schema-aligned JSON — using `Qwen2.5:32b` via Ollama for free-text fields and deterministic HTML table parsing for structured tables.

A Streamlit-based evaluation frontend allows side-by-side comparison of extraction results against ground truth and GPT-based baselines, generating color-coded Excel reports with field-level `MATCH`, `MISMATCH`, `NEAR_MATCH`, and `MISSING_KEY` analysis.

The system requires zero proprietary APIs, runs entirely on local hardware, and achieves 85–90% match rates while maintaining 100% accuracy on rule-based table extraction. On purchase orders, the local pipeline outperforms GPT by 26 percentage points. Adding new document types requires zero changes to any core pipeline file.

Before arriving at this design, six prior approaches were prototyped and evaluated — Donut, Tab2DB, LLMware, Granite-only, GLM-OCR + Granite, and GLM-OCR + Qwen2.5. Chapter 3 documents the full findings from each, which collectively informed the architectural decisions of the final pipeline presented from Chapter 4 onwards.

2 Introduction

2.1 Problem Statement

Commercial documents—purchase orders, shipping bills, invoices—arrive as PDF files, but downstream enterprise systems require structured data. Manually keying in hundreds of fields across dozens of pages is:

- **Slow** — a single purchase order with 24 line items across 14 pages takes 30–60 minutes to key in manually.
- **Error-prone** — manual data entry introduces transcription errors, especially in numeric fields like quantities, prices, and dates.
- **Expensive** — at scale, this requires dedicated data entry teams costing significant operational overhead.
- **Privacy-sensitive** — cloud-based extraction services require uploading confidential commercial documents containing pricing, supplier terms, and trade secrets to third-party servers.

Existing solutions either rely on proprietary cloud APIs (raising data privacy concerns for sensitive commercial documents), require extensive per-document training data, or fail on complex table layouts that span multiple pages—a common pattern in commercial documents.

2.2 Aim of the Project

To build a **fully local, end-to-end pipeline** that takes a raw PDF document as input and produces a clean, schema-aligned JSON file as output—without any cloud dependency, proprietary APIs, or API keys—while providing an intuitive evaluation interface for quality assessment.

2.3 Objectives

1. Deliver fast, reliable, and user-friendly extraction of structured data from complex commercial PDF documents with minimal human intervention.
2. Produce clean key–value pair JSON output that is schema-aligned, validated, and ready for direct consumption by downstream enterprise systems.
3. Ensure high extraction accuracy across both free-text fields (names, dates, addresses) and structured tabular data (line items, shipment schedules, pricing).
4. Build a fully local, privacy-preserving solution that processes confidential documents entirely on-premises with zero cloud dependency.
5. Design a generic and extensible architecture that supports new document types without modifying the core extraction engine.
6. Provide an intuitive evaluation interface that enables non-technical users to visually assess and compare extraction quality through downloadable reports.

2.4 Scope of the Project

In Scope

- OCR extraction from PDF/image documents using GLM-OCR (0.9B params)
- Layout detection using PP-DocLayout-V3 (25 label categories)
- Structured data extraction using Qwen2.5:32b (local) + rule-based parsing
- Two document types: Purchase Order (Li & Fung) and Shipping Bill (Indian Customs)
- Streamlit evaluation frontend with 3-way comparison (GT vs MyModel vs GPT)
- Color-coded Excel report generation with per-field status tracking
- Extensible plugin architecture for new document types

Out of Scope

- Handwritten document recognition
- Real-time streaming extraction
- Cloud deployment or SaaS packaging
- Training or fine-tuning of OCR/LLM models
- Languages other than English

3 Experiments

Before the final pipeline was designed, **six prior approaches** were prototyped and evaluated end-to-end on real Shipping Bill and Purchase Order documents. The work proceeded in two phases. **Phase I** consisted of *three independent experiments* with existing extraction frameworks (Donut, Tab2DB, LLMware), each evaluated as a standalone candidate. After Phase I established that no off-the-shelf framework was a complete solution, the extraction problem was decomposed into three independent sub-problems (see Section 3.4), and **Phase II** ran three further experiments (Granite-only, GLM-OCR + Granite, GLM-OCR + Qwen2.5) as a *connected chain* addressing one of those sub-problems incrementally.

This chapter documents the full findings from every experiment — architecture, models used, license, pros, cons, our results, production-safety, what's missing, and future scope — so that future engineers inheriting this problem start from a documented research baseline.

Phase I — Three Independent Experiments

3.1 Donut

`Donut` (clovaai/donut) is an OCR-free Document Understanding Transformer that takes a document page image and emits structured output directly, with no intermediate OCR stage. It was evaluated as the most ambitious end-to-end candidate.

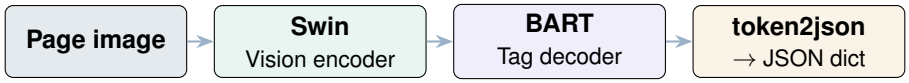


Figure 3.1: Donut architecture — OCR-free vision-encoder-decoder (Swin Transformer encoder + BART-style decoder)

What it does

1. Takes a document page image → generates structured output (fields/tables) without OCR.
2. Uses a vision encoder + text decoder to generate a tag/markup sequence, then converts it to JSON-like dict (e.g., via `token2json`).
3. Works well only when the page looks like what it was fine-tuned on (e.g., receipts/invoices); otherwise outputs become off-schema / unreliable.

Models Used

The pipeline uses the `Donut` architecture, which combines a Swin Transformer encoder with a BART-style decoder for OCR-free document understanding. Public checkpoints are available for invoice and receipt understanding tasks, including `naver-clova-ix/donut-base-finetuned-cord-v2`.

Model	Donut
Architecture	Swin Transformer encoder with BART-style decoder
Public Checkpoints	Available for invoice and receipt understanding tasks
Repository	Open source
Model Availability	Open source architecture with multiple public weights
Suitability	Not production-safe without task-specific validation and hardening

Pros

1. End-to-end (no OCR step).
2. Strong for tasks it's fine-tuned for.
3. Lots of community models / examples.

Cons

1. Works well only on the document type it was trained on.
2. Quality drops with domain mismatch (e.g., fiscal tables).
3. Needs schema/prompt discipline.
4. Requires fine-tuning + strict validation.

Our results

1. Ran the end-to-end Donut pipeline successfully, but the extracted output quality was poor because we used a CORD-v2 (receipt) fine-tuned checkpoint that doesn't match Shipping Bill / customs layouts or our SB schema.
2. To make Donut useful for our use case, it needs domain fine-tuning on our schema.
3. Not production-safe as a generic "works on every PDF" solution: without domain training, Donut won't generalize reliably across new templates, so we'd end up fine-tuning again for each new document family.
4. Unreliable as a primary extractor, but can still be used as a baseline / quick prototype (or as a supporting signal) to compare against Tab2DB-style pipelines and measure improvements after fine-tuning.

What's missing

1. Full-section coverage.
2. Multi-row table extraction — instead of capturing the complete items table, Donut picked one "clean-looking" line item and skipped the rest (table fidelity issue).
3. Schema-faithfulness — values are getting forced into our SB schema, but the structure looks "receipt-like" (CORD bias), so fields can be mis-mapped / drifted.
4. Document-level consistency — page/id signals look inconsistent (the same leading id like `6` appeared across rows), meaning the model is not reliably tying outputs to the correct page/record.
5. Generalization — this setup won't work "for every PDF template" because the pre-trained CORD checkpoint doesn't understand Shipping Bills the way it understands receipts.

Future scope

1. Fine-tune on our fiscal/budget table schema (a few hundred labeled pages) to handle domain mismatch.
2. Add PDF → image rendering + layout-aware cropping/tiling so tables are fed cleanly (improves stability a lot).
3. Add strict schema validation + numeric consistency checks (totals, hierarchy) with retry/repair.

3.2 Tab2DB

`Tab2DB` (xKDR's `LLM_table2db`) takes a long fiscal PDF, renders pages to images, and runs an LLM page-by-page with a fixed schema prompt to produce structured CSV / JSON outputs, while carrying state across pages for continuation tables.

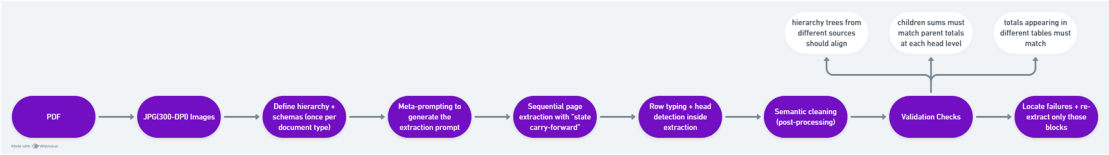


Figure 3.2: Tab2DB — the actual nine-stage implemented pipeline with multi-level validation and selective re-extraction.

What it does

1. Takes a long fiscal PDF (hundreds of pages) and renders pages to images (because the text layer can be broken / multilingual).
2. Runs an LLM page-by-page with a fixed schema prompt to extract structured outputs (CSV/JSON-style tables).
3. Carries context across pages (keeps previous-page headers/keys) so continuation tables don't lose structure.
4. Applies validation + consistency checks (within-table totals, cross-level totals) to catch wrong extractions.
5. Writes clean, schema-aligned CSVs per section/type so the output is usable for downstream DB / analytics.

Models Used

The extraction pipeline in the paper uses `Gemini 2.5 Pro`, a closed-source large language model. Since the model is proprietary, it does not provide local deployment support or full control over inference behavior, which limits direct production adoption in a fully local extraction pipeline.

Model	Gemini 2.5 Pro
Model Type	Closed-source large language model
Local Deployment	Not supported
Inference Control	Limited due to proprietary access
Suitability	Limited for fully local production pipelines

Pros

1. Good fit for fiscal PDFs — built for large government-style budget documents (hierarchy + totals).
2. Page-to-page context + checks — carries context across pages and uses validation / consistency checks to reduce obvious mistakes.
3. Schema-first output (CSV / structured) — outputs structured tables that are easier to load into DB than raw text.

Cons

1. Manual prompt / schema work — still needs prompt engineering; missing meta-prompting (auto schema + auto prompt generation + reuse).
2. Model dependency / cost risk — the paper used Gemini 2.5 Pro (closed/paid); switching models can change quality and require re-tuning.
3. Repo brittleness — often has hard-coded assumptions; needs refactor/config + retries / logging / QA to generalize across new PDFs.

Our results

Not satisfactory — although the model extracted text and numbers, it couldn't reliably map them into our target CSV schema. The pipeline depends heavily on manual prompt + manual schema (no real "learning / adaptation" to our PDF structure), so outputs were misaligned with columns/fields and required too much manual correction. *Gemini was only filling values into our contract, not defining the contract.*

What's missing

1. Meta-prompting (auto schema + auto prompt generation).
2. A "few pages input → generate new prompt for this PDF" stage.
3. Persisting a newly generated prompt and reusing it for remaining pages.

Future scope

1. Build a meta-prompting / agent step — auto-detect schema from a few pages → generate prompt automatically.
2. Improve reliability with structured-output enforcement (column-count / type checks), retries, and post-processing to map fields correctly.
3. Make it production-ready via a config-driven pipeline (no hardcoding), logging / QA dashboards, and model-agnostic backends (swap Gemini → open models).

3.3 LLMware

`LLMware` is a retrieval-augmented document framework where the LLM is one component among several — a vision-language reader, an embedding model, a vector database, and a structuring LLM.

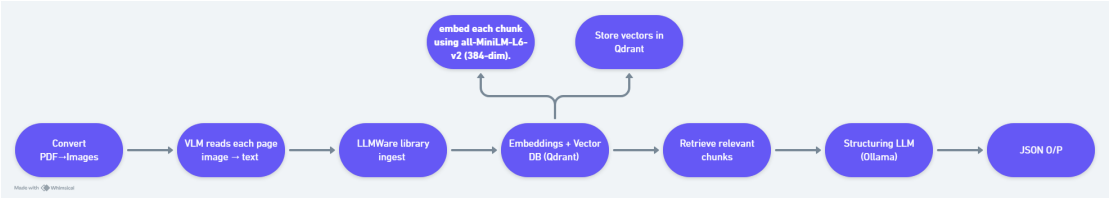


Figure 3.3: LLMware — the actual seven-stage implemented pipeline with separate embedding, vector store, and retrieval components

What it does

1. Takes a PDF → converts pages to images → reads each page (VLM/OCR) into text, then LLMware ingests + chunks it.
2. Embeds each chunk (e.g., `all-MiniLM-L6-v2`) and stores vectors in Qdrant, so you can retrieve only the relevant chunks for a query/section.
3. Feeds retrieved chunks to a "structuring" LLM (e.g., Ollama model) that outputs schema-aligned JSON (key-values + tables/arrays) instead of dumping raw text.

Models Used

The LLMware-based pipeline used a multi-component local stack consisting of a vision-language page reader, an embedding model for retrieval, a vector database, and a structuring LLM. The initial setup used `vision-qwen2.5:7b-instruct` for page reading, `embedding-all-MiniLM-L6-v2` for 384-dimensional embeddings with Qdrant, and `structure-llama3:latest` for JSON structuring.

A second iteration upgraded the reader to `Qwen2-VL-2B-Instruct` and the structurer to `qwen2.5:14b-instruct`, along with a schema-aware validate-and-merge step.

Pipeline Type	Retrieval-augmented document extraction pipeline
Page Reader	`vision-qwen2.5:7b-instruct`; later upgraded to `Qwen2-VL-2B-Instruct`
Embedding Model	`embedding-all-MiniLM-L6-v2` with 384-dimensional embeddings
Vector Store	Qdrant
Structuring Model	`structure-llama3:latest`; later upgraded to `qwen2.5:14b-instruct`
Repository	Open source
Model Availability	Open source models used
Production Readiness	Not yet production-safe without stronger validation and table-reconstruction checks

Pros

1. All-in-one doc pipeline (parse → chunk → retrieve → generate).
2. Works with many models / backends (no lock-in).
3. Good for long PDFs via chunking + vector search.
4. Supports structured outputs (easy to produce JSON with a schema prompt).

Cons

1. Not plug-and-play for "any PDF → perfect JSON" (needs pipeline setup).
2. Tables are still hard (layout / OCR issues can break rows / columns).
3. Quality depends on chosen models (swap models = different results).
4. Needs strong validation / repair to be production-safe (schema checks, retries).

Our results

1. We got the end-to-end LLMware pipeline running successfully: document → chunking / retrieval → structured JSON generation.
2. We used the stack: `qwen2.5:7b-instruct` (vision/OCR) + `all-MiniLM-L6-v2` (384-dim, Qdrant) + `llama3:latest` (structuring).
3. It did produce JSON outputs, but field mapping was inconsistent across pages/sections.
4. Tables / line-items were the weakest part: extraction and alignment were unstable, so results weren't production-safe yet.

What's missing

1. Reliable table reconstruction (multi-line rows, merged cells, continuation across pages).
2. Validation + repair loops to fix inconsistent fields and retry weak sections.
3. Normalization / post-processing (dates, amounts, codes, units) to make outputs consistent.
4. A proper evaluation harness (field/table metrics + regression tests) to compare runs / models.

Future scope

1. Add a table-specialized step before/after structuring to improve row / column fidelity.
2. Introduce auto-check + retry on low-confidence extractions (page/section-level).
3. Build post-processing utilities for standardizing numbers / dates and cleaning noise.
4. Create a benchmark suite to track improvements across models and PDFs.

Sub-Problem Decomposition — bridge between Phase I and Phase II

3.4 Sub-Problem Decomposition

After Phase I established that no off-the-shelf framework was a complete solution, the extraction problem was **decomposed into three independent sub-problems**, each addressed as its own line of work. The three sub-problems are themselves independent of one another — a candidate solution to one does not constrain the candidates for the others.

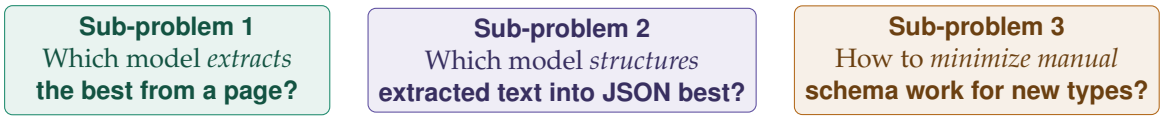


Figure 3.4: The three independent sub-problems that Phase II addressed

Sub-problem 1 — OCR / page reading. Evaluated multiple OCR engines on real documents (complex tables, seals, formulas). `GLM-OCR` (0.9B parameters, served via vLLM) was selected because it produced clean, layout-preserving text including HTML tables with correct column counts.

Sub-problem 2 — structuring. Approached as a chain of three connected experiments (Sections 3.5–3.7). This is the only chain in this report.

Sub-problem 3 — minimizing manual schema work. The objective here was *minimization*, not full automation. A pragmatic plugin architecture was designed in which the schema for a document family is captured as three artifacts (null template + config + adapter), authored once per family, and consumed by an unchanged generic engine.

Phase II — Connected Experiments for Sub-Problem 1 & Sub-Problem 2

3.5 Granite-only

The first experiment within Sub-problem 2 was to use a single multimodal model for both reading the page and structuring the result. `granite3.2-vision:2b` was tested as a one-stop reader-plus-mapper.

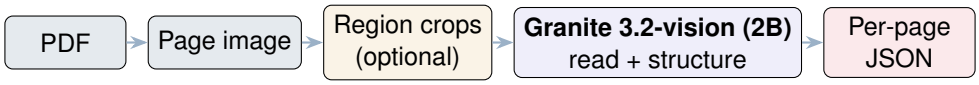


Figure 3.5: Granite-only — one small multimodal model attempts both reading and structuring

What it does

1. Converts PDF → page images (and optionally crops key regions like header / table / footer) to make extraction easier.
2. Extracts raw text from each page / region (either directly via Granite vision or via an OCR-first stage, depending on the run).
3. Structures the extracted content into JSON-like output (Granite used mainly as the structuring / mapping model).
4. Saves artifacts for evaluation: page images / crops, raw text outputs, per-page structured JSON, and combined JSON for all pages.

Models Used

The Granite-only experiment used `granite3.2-vision:2b`, a small open-weights multimodal model, as a single-stage document understanding system. In this setup, the model was responsible for both reading the page image and structuring the extracted content into JSON-like output.

Model	`granite3.2-vision:2b`
Model Type	Multimodal vision-language model
Pipeline Role	Single-stage page reading and JSON structuring
License	Apache-2.0
Repository	Open source
Model Availability	Open source model weights available
Production Readiness	Not production-safe for dense commercial PDFs without stronger OCR, validation, and table handling

Pros

1. Lightweight (2B) and fast to run — a good trade-off for quick document / image understanding on limited GPUs / CPU.
2. Multimodal (vision + text) — can read page images (forms, receipts, charts/tables) without needing a separate OCR tool in many cases.
3. Open license (Apache 2.0) — easier to use in research / industry workflows with fewer licensing headaches.
4. Easy to integrate — works well as a "document understanding" component in pipelines (extract → structure → validate).

Cons

1. Struggles on dense / complex tables — row / column fidelity drops, leading to missing / merged content.
2. Layout sensitivity — multi-block pages (tables + stamps + footers) can cause partial extraction or repeated fragments.
3. Not guaranteed strict JSON — needs schema enforcement / validators; otherwise output can drift or hallucinate fields.
4. Image-quality dependent — small fonts / low contrast / scans require preprocessing or OCR-first fallback for reliability.

Our results

1. Granite (no cropping) captured some headings / key fields, but overall raw text quality was unstable.
2. The output was very noisy and repetitive, with the same content blocks repeated multiple times.
3. Some text appeared garbled / merged, reducing correctness and breaking clean parsing.
4. Because raw text was unreliable and layout / table structure wasn't preserved, the downstream JSON-like output was poor and inconsistent.
5. Increasing DPI did not materially improve extraction stability / structure.

What's missing

1. Remove repeated blocks (de-dup cleanup) before JSON mapping.
2. Split page into sections (header / parties / table / totals) so text doesn't mix.
3. Table-aware extraction to preserve rows / columns for correct JSON.
4. Clean up noisy / garbled text (normalization).

Future scope

For this pipeline to work reliably we likely need a cleaner text extraction stage first — ideally a table-aware OCR / VLM or a conversion-first model that can produce stable, non-duplicated text from complex layouts. Once we have clean raw text, we can then properly evaluate the second-stage performance for mapping / structuring into JSON (schema enforcement, key-value mapping, validations, etc.).

Lesson Carried Forward

A small multimodal model cannot reliably do both reading and structuring at once. We need a dedicated OCR stage in front of any structuring model. **This lesson directly motivated the next experiment (GLM + Granite).**

3.6 GLM + Granite

Splitting the OCR job out: `glm-ocr:latest` reads the page, `granite3.2-vision:2b` structures the OCR text into JSON.

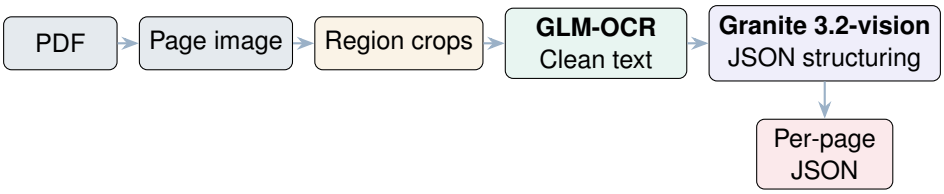


Figure 3.6: GLM + Granite — dedicated OCR feeds dedicated structurer

What it does

1. Converts PDF → page images (and optionally crops key regions like header / table / footer).
2. GLM performs OCR on the page / crops to extract clean raw text.
3. Granite maps / structures that extracted text into JSON-like output (key fields, sections, table text).
4. Saves artifacts for evaluation: page images / crops, raw OCR text, structured JSON, and combined outputs (single-page + all-pages).

Models Used

The GLM + Granite experiment used a two-stage local pipeline. `glm-ocr:latest` was used as the dedicated OCR layer to extract clean raw text from document images, while `granite3.2-vision:2b` was used as the structuring model to map the extracted OCR text into JSON-like output.

OCR Model	`glm-ocr:latest`
Structuring Model	`granite3.2-vision:2b`
Pipeline Type	Two-stage OCR-first extraction pipeline
GLM-OCR License	MIT
Granite License	Apache-2.0
Repository	Open source
Model Availability	Open source models used
Production Readiness	Not yet production-safe; structuring consistency and table reconstruction require further hardening

Pros

1. Cleaner input → better results — GLM extracts clean, readable text, so Granite gets strong input instead of struggling with page reading.
2. Granite focuses on structuring — with OCR handled, Granite can concentrate on formatting + organizing into JSON.
3. Modular + debuggable — clear separation of OCR vs mapping makes it easy to pinpoint failures and iterate faster.
4. More robust via confidence workflow — if OCR looks weak, you can re-run with higher DPI or switch OCR / model without changing the whole pipeline.

Cons

1. Structuring is still error-prone — the mapper can mis-assign values to wrong keys if the text is ambiguous.
2. Tables are difficult — OCR often flattens tables, and rebuilding true row / column structure in JSON is hard.
3. Schema consistency isn't guaranteed — without strict schema + validation, JSON can drift (missing keys, inconsistent nesting / types).
4. Sensitive to OCR quality / segmentation — noisy text, repeated labels, or poor section boundaries can cause missing fields or hallucinated fills.

Our results

1. GLM OCR performed well and produced clean, readable raw text on most pages, making the extraction stage reliable.
2. With that OCR text, Granite worked better as a "structuring / mapping" model than it did as a direct page reader.
3. The main remaining issue was JSON-mapping consistency — Granite sometimes produced unstable key–value assignments and inconsistent structure.
4. Table-heavy sections were still the hardest — OCR text from tables often came flattened, so Granite struggled to rebuild correct row / column structure in JSON.
5. The confidence-driven workflow (re-run low-quality pages with higher DPI / fallback) helped keep extraction usable, but didn't fully solve table-to-JSON correctness.

What's missing

1. Granite JSON mapping remained inconsistent — even with good OCR, Granite sometimes produced incorrect or unstable key–value assignments.
2. Table-heavy pages were hardest — OCR gives flattened table text, and Granite struggled to reconstruct accurate row / column structure in JSON.
3. Schema drift / missing keys — output JSON could vary across pages (missing fields, inconsistent nesting, or formatting changes).
4. Hallucinated / missing fields — Granite may fabricate values to complete the JSON or drop fields when the OCR text is incomplete or poorly segmented.

Future scope

1. Try a different model for JSON mapping — keep GLM as the OCR layer (since it performed well), but replace Granite with a model better suited for structured extraction / function calling / strict JSON.
2. Improve table handling — add a table-aware step (table-structure detection / row-column reconstruction) before sending content to the JSON mapper.

Lesson Carried Forward

GLM-OCR is the right OCR model — a settled component going forward. The remaining weakness is the structurer: it needs to map clean text to JSON consistently, and tables need a deterministic path. **This lesson directly motivated the next experiment (GLM + Qwen).**

3.7 GLM + Qwen

Same OCR layer (GLM) as before, but the structurer is replaced with a stronger open-weights model: `qwen2.5:14b-instruct`. A small benchmarking framework also compared seven structurer candidates head-to-head on the same OCR output.

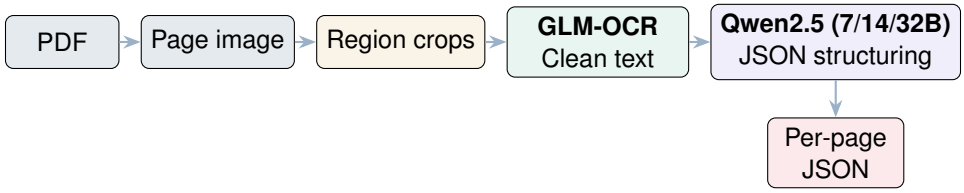


Figure 3.7: GLM + Qwen — identical structure to GLM + Granite, with a stronger structurer

What it does

1. Convert PDF → page images.
2. Apply a cropping strategy to focus on relevant regions (e.g., header blocks / table regions / footer).
3. Use GLM-OCR to extract clean raw text from the cropped regions.
4. Use Qwen (`qwen2.5:14b-instruct`) to map / structure that extracted text into structured JSON (no strict schema forcing / no hard key-mapping).

Models Used

The GLM + Qwen experiment used a two-stage local pipeline. `glm-ocr:latest` was used as the dedicated OCR layer for extracting clean raw text from document images, while `qwen2.5:14b-instruct` was used as the primary structuring model to convert OCR output into schema-aligned JSON.

The benchmarking harness also evaluated multiple alternative structuring models, including `qwen2.5:7b-instruct`, `qwen2.5:32b-instruct`, `mixtral:8x7b`, `llama3.1:latest`, `mistral:latest`, and `ministral-3:8b`.

OCR Model	`glm-ocr:latest`
Structuring Model	`qwen2.5:14b-instruct`
Benchmark Models	`qwen2.5:7b-instruct`, `qwen2.5:32b-instruct`, `mixtral:8x7b`, `llama3.1:latest`, `mistral:latest`, `ministral-3:8b`
Pipeline Type	Two-stage OCR-first extraction and JSON-structuring pipeline
GLM-OCR License	MIT
Qwen2.5 License	Apache-2.0
Repository	Open source
Model Availability	Open source models used
Production Readiness	Not yet production-safe; table reconstruction, schema enforcement, and validation checks require further hardening

Pros

1. Clean extraction + strong structuring — GLM gives readable text; Qwen (14B) is strong at organizing it into consistent JSON.
2. Works well on messy real PDFs — OCR-first handles scans / stamps better than pure VLM prompting in many cases.
3. Modular + debuggable — you can fix OCR issues and mapping issues separately (faster iteration).
4. Scales to multi-page documents — the same OCR → JSON loop runs per page, then merges results.

Cons

1. Tables are still the hardest — OCR often flattens tables; even a strong LLM can't perfectly reconstruct rows / columns reliably.
2. Mapping depends on OCR quality — any OCR noise, wrong reading order, or missing lines directly hurts JSON correctness.
3. Not "strict JSON" by default — Qwen may drift from schema, omit keys, or invent values unless schema + validations are enforced.
4. Cost / latency — 14B mapping is heavier (slower / more VRAM) than smaller mappers, especially across many pages.

Our results

1. GLM performed well at producing clean, readable raw text, which gave Qwen a stable input for structuring.
2. Qwen produced structured JSON more consistently, with better formatting and more stable key–value organization than doing OCR + JSON in one step.
3. Complex tables were still the hardest — OCR flattened table content and Qwen couldn't reliably reconstruct perfect row / column structure in JSON.
4. Ambiguous multi-block pages sometimes led to incorrect key assignments, and JSON format could vary across pages without strict enforcement.

What's missing

1. Table-aware extraction / reconstruction so rows / columns don't get flattened before JSON mapping.
2. Post-processing cleanup (normalize OCR text, remove duplicates, fix line breaks) before sending to Qwen.
3. Quantitative evaluation (coverage %, field accuracy, table fidelity) to measure improvements objectively.

Future scope

1. Try a stronger JSON-mapping model (compare Qwen variants / other LLMs) to improve key–value accuracy and reduce drift.
2. Add a table-specific extraction step (row / column reconstruction) so item tables convert reliably into structured output.
3. Improve OCR preprocessing (de-skew, contrast / sharpening, noise removal) to boost text quality on scanned / low-quality pages.
4. Add normalization + cleanup before mapping (remove repeated blocks, fix line breaks, standardize numbers / dates).
5. Build an evaluation benchmark (field-level accuracy, table fidelity, page coverage) to compare models and track progress.

Lesson Carried Forward

A strong open-source structurer like Qwen2.5 is sufficient for free-text fields, but tables need a deterministic, rule-based path on top of OCR. The right architecture is a **hybrid**: AI for natural language understanding, rules for structured tabular data. **This is the design inherited by the final pipeline (Chapter 4).**

3.8 Summary of Findings

The combined Phase I + Phase II investigation yielded the architectural decisions which the final pipeline (Chapter 4) inherits directly:

#	From	Decision in the final pipeline
1	Donut	Schema lives outside the model, as a swappable artifact
2	Tab2DB	Per-page extraction with strict schema; previous-page state for continuation
3	LLMware	Reading and structuring as separate components, but no RAG between them
4	Sub-problem 1	GLM-OCR + PP-DocLayout-V3 as the dedicated reading layer
5	GLM + Qwen	Strong open-weights structurer for free text + rule-based HTML parser for tables
6	Sub-problem 3	Plugin architecture: null template + config + adapter, registered once per family

Table 3.1: Mapping from experiment findings to final-pipeline decisions

4 Final Workflow

4.1 Workflow Overview

The pipeline operates in two stages connected by a one-time manual schema design step. The overall flow is: Raw PDF → OCR Extraction → Schema Design → Structured Extraction → JSON Output → Evaluation.

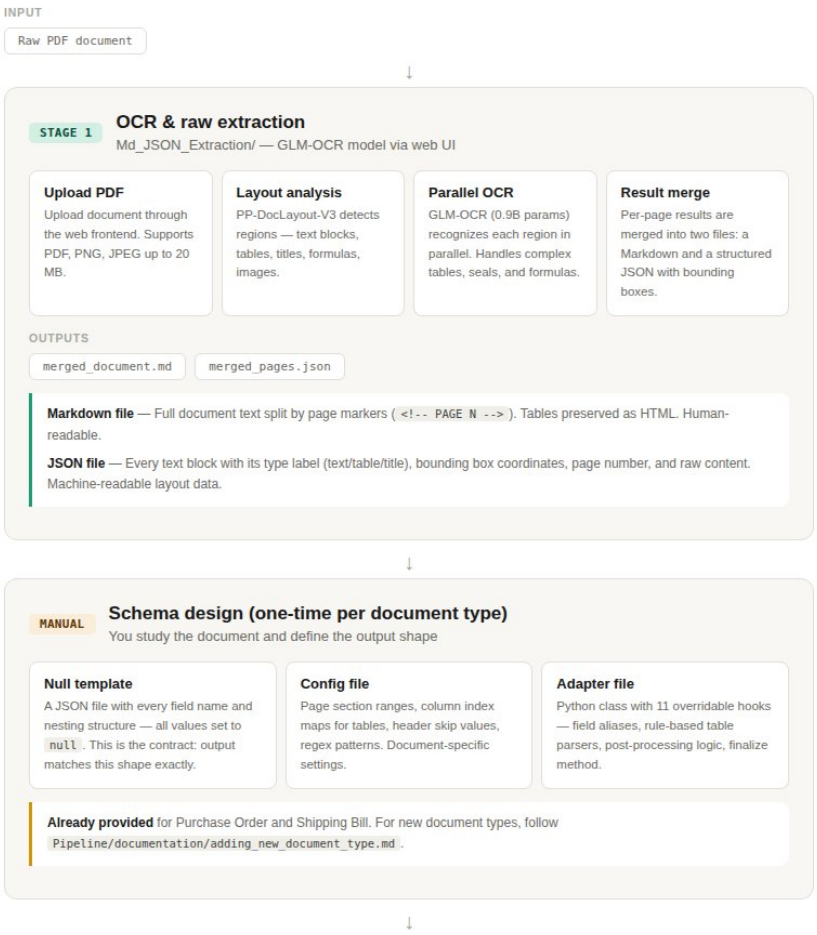


Figure 4.1: Stage 1 OCR & raw extraction and the one-time manual schema design step

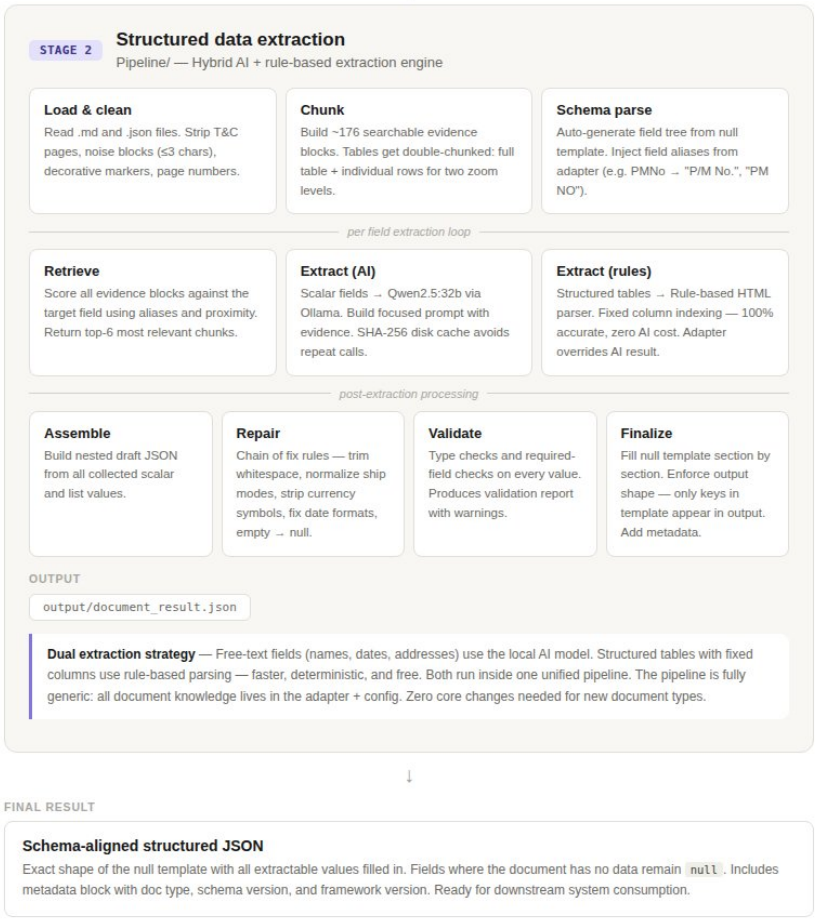


Figure 4.2: Stage 2 structured data extraction with hybrid AI + rule-based engine, producing the final schema-aligned JSON

4.2 The 11-Step Extraction Pipeline

#	Stage	Description
1	Load Markdown	Split by `PAGE` markers, strip Terms & Conditions pages
2	Load JSON	Unwrap OCR blocks per page, handle double-wrapping
3	Clean Inputs	Strip noise blocks (≤ 3 chars), page markers, decorative lines
4	Build Evidence	Merge into ∼176 searchable chunks; tables double-chunked (full + per-row)
5	Parse Schema	Auto-generate field tree from null template, inject field aliases
6	Extract Scalars	Per field: retrieve top-6 chunks → SHA-256 cache check → Qwen2.5 prompt
7	Extract Lists	Rule-based HTML table parser overrides AI for structured tables
8	Assemble	Build nested draft JSON from all collected scalar and list values
9	Repair	Fix whitespace, normalize dates / currency / ship modes, empty → `null`
10	Validate	Type checks (string, number, list) and required-field checks
11	Finalize	Fill null template section by section, enforce output shape, add metadata

Table 4.1: The 11-step extraction assembly line

4.3 Dual Extraction Strategy

The pipeline uses two extraction methods and intelligently picks the right one per field:

Method	Used for	How it works	Accuracy	Speed
AI	Free-text fields	Retrieve chunks → build prompt → Qwen2.5	High	2–5 s
Rules	Structured tables	HTML parser reads `<td>` by column index	100%	<100 ms

Table 4.2: Dual extraction strategy comparison

4.4 Project Features

- **Fully Local & Private** — All processing happens on-premises. No document data ever leaves the machine, ensuring complete confidentiality of commercial documents.
- **Dual Extraction Engine** — AI handles free-text; rules handle tables. Best of both worlds.
- **User-Friendly Evaluation UI** — A clean Streamlit web interface lets users upload files, run comparisons, and download reports with a few clicks — no command-line knowledge required.
- **Generic Plugin Architecture** — The core pipeline never changes. Adding a new document type = 3 files + 1 line in the registry. Future-proof and maintainable.
- **Intelligent Caching** — SHA-256 disk-backed response cache ensures re-runs and debugging iterations are near-instant.
- **Color-Coded Excel Reports** — Downloadable Excel files with green / yellow / orange / red row coloring for instant visual assessment of extraction quality.
- **3-Way Comparison** — Compare Ground Truth vs Your Pipeline vs GPT side-by-side in a single report.
- **Per-Page & Overall Recall** — The evaluation engine computes field-level overlap recall both per-page and across the entire document.

4.5 Design Diagram — Layered Architecture

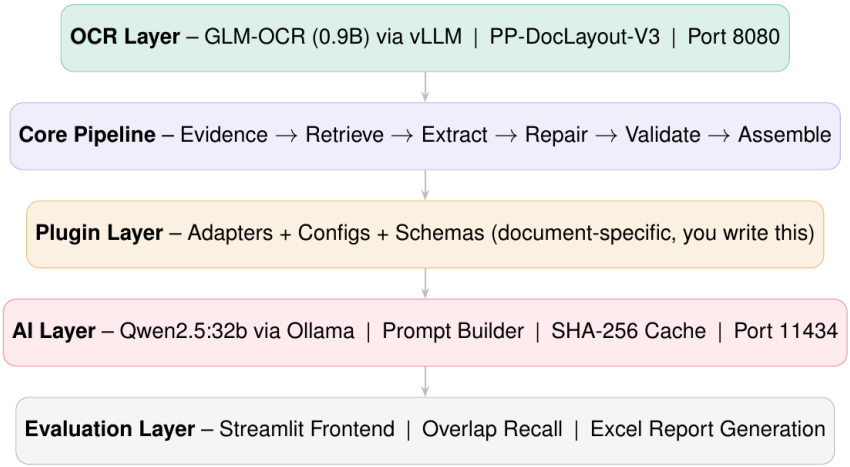


Figure 4.3: Five-layer system architecture

4.6 Design and Implementation Constraints

- All processing must run entirely offline — no external API calls permitted at any stage.
- Extraction must be deterministic and reproducible when cache is enabled.
- Output JSON must match the null template shape exactly — no extra or missing keys allowed.
- Core pipeline files must never be modified when adding new document types.
- Rule-based parsing is preferred over AI wherever column positions are fixed and known.
- The evaluation frontend must work without requiring any programming knowledge from the user.

4.7 Assumptions and Dependencies

Item	Details
GPU availability	24 GB+ VRAM recommended for serving both GLM-OCR (vLLM) and Qwen2.5 (Ollama)
Document structure	Documents follow a consistent layout within each type (column positions are fixed)
OCR quality	GLM-OCR produces accurate HTML tables with correct column counts
Ollama server	Must be running on port 11434 for AI extraction calls
vLLM server	Must be running on port 8080 for OCR inference
Python 3.12+	Required for GLM-OCR SDK and UV package manager

Table 4.3: Key assumptions and dependencies

5 Evaluation Frontend

The project includes a Streamlit-based evaluation frontend that provides a user-friendly web interface for assessing extraction quality. No command-line knowledge is required — everything is point-and-click.

5.1 Frontend Architecture

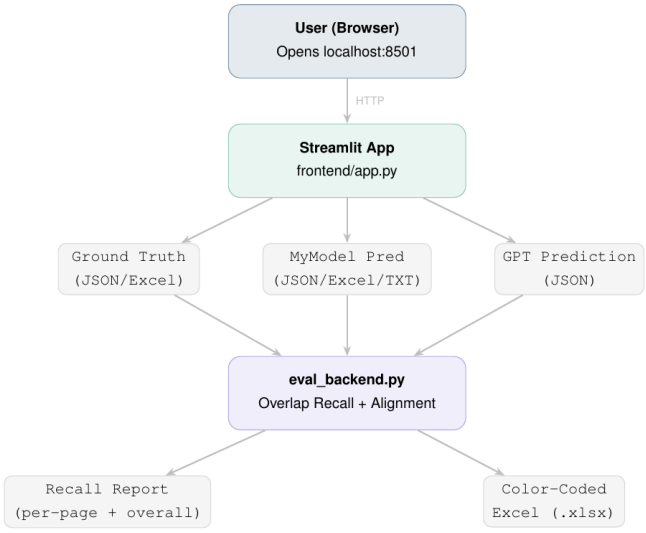


Figure 5.1: Evaluation frontend — component diagram

5.2 User Interface

The frontend presents a clean, branded interface with the Bosch logo, a dark sidebar for file uploads, and action buttons for running evaluations.

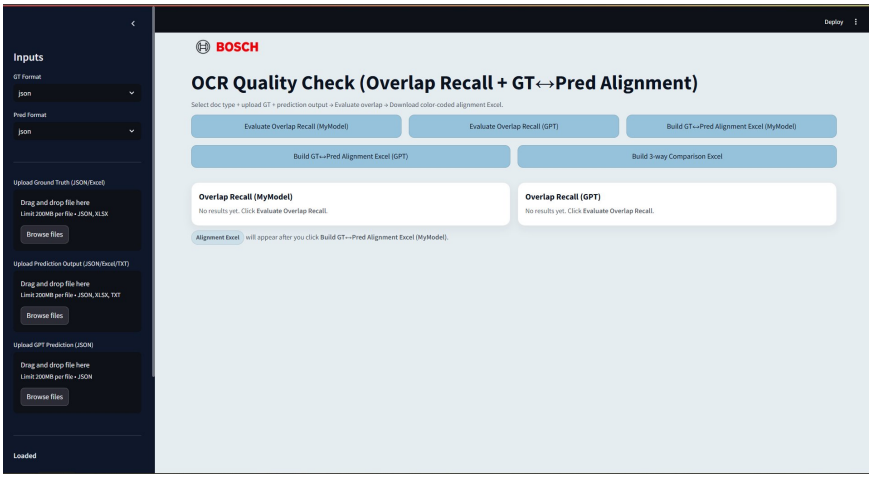


Figure 5.2: Streamlit evaluation frontend — main interface

5.3 Input Files

The user uploads three files through the sidebar:

Upload Slot	Format	What it contains
Ground Truth	JSON or Excel	The expected correct values for every field in the document
MyModel Prediction	JSON, Excel, or TXT	The structured JSON output produced by our pipeline (Stage 2)
GPT Prediction	JSON	The extraction output produced by GPT for comparison

Table 5.1: Three input files for evaluation

5.4 Evaluation Actions

Five action buttons are available:

1. **Evaluate Overlap Recall (MyModel)** — Computes per-page and overall field-level recall for the local pipeline output.
2. **Evaluate Overlap Recall (GPT)** — Same metric computed for the GPT extraction output.
3. **Build GT↔Pred Alignment Excel (MyModel)** — Generates a color-coded Excel file comparing ground truth against the local pipeline.
4. **Build GT↔Pred Alignment Excel (GPT)** — Same Excel report for the GPT output.
5. **Build 3-Way Comparison Excel** — A single Excel file comparing Ground Truth vs MyModel vs GPT side-by-side.

5.5 Excel Report Output

The generated Excel report provides a field-by-field comparison with five columns: GT Key, Pred Key, GT Value, Pred Value, and Status. Each row is color-coded based on the match status.

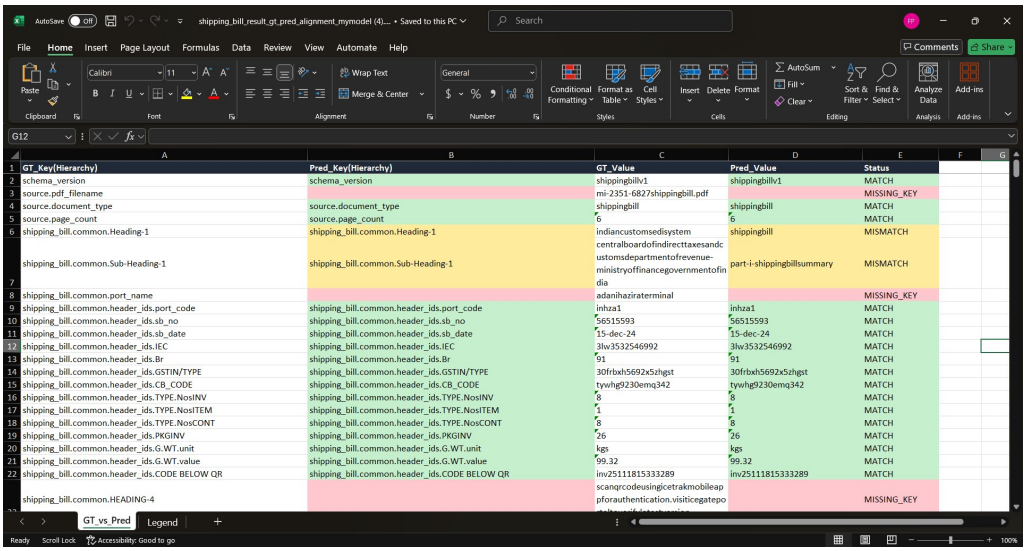
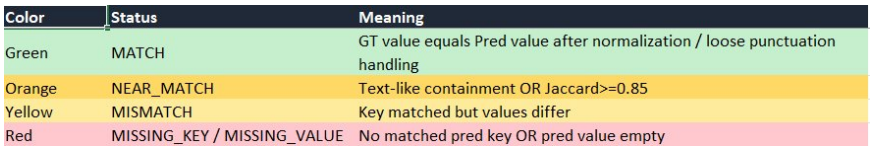


Figure 5.3: Color-coded Excel alignment report — GT vs Predicted values

5.6 Status Legend

Each extracted field in the comparison report is assigned one of four evaluation statuses. These statuses indicate whether the predicted value matches, partially matches, differs from, or is missing with respect to the ground-truth value.



Color	Status	Meaning
Green	MATCH	GT value equals Pred value after normalization / loose punctuation handling
Orange	NEAR_MATCH	Text-like containment OR Jaccard>=0.85
Yellow	MISMATCH	Key matched but values differ
Red	MISSING_KEY / MISSING_VALUE	No matched pred key OR pred value empty

Figure 5.4: Color-coded status legend used in the Excel comparison report.

Color	Status	Definition
Green	**MATCH**	Ground-truth and predicted values are equal after normalization and loose punctuation handling.
Orange	**NEAR_MATCH**	Values are not identical, but show strong similarity based on text containment or Jaccard token similarity ≥ 0.85.
Yellow	**MISMATCH**	The key is present in the prediction, but the extracted value differs from the ground-truth value.
Red	**MISSING_KEY**	No corresponding prediction key was found, or the predicted value is empty.

Table 5.2: Detailed definitions of field-level comparison statuses.

6 Project Life Cycle

6.1 Development Methodology

The project followed an iterative, problem-decomposition approach inspired by Agile principles. Rather than treating the entire pipeline as a monolithic problem, the core challenge was broken down into three independent sub-problems, each tackled as a separate research-and-build sprint. This allowed rapid experimentation within each sub-problem while maintaining a clear integration path.

6.2 Why This Approach

Each sub-problem was independently testable — the OCR model could be evaluated on its own using benchmark documents before any structuring pipeline existed. The structuring model could be tested with manually prepared OCR outputs before the OCR stage was finalized. This decoupling reduced risk and allowed parallel experimentation.

Within each sprint, the workflow was iterative:

1. **Research** — Survey available models and tools for the sub-problem.
2. **Prototype** — Build a minimal working version with sample documents.
3. **Evaluate** — Measure accuracy against ground truth.
4. **Iterate** — Tune configurations, add repair rules, improve aliases until fill rate plateaus.
5. **Integrate** — Connect the sprint output to the next sub-problem.

6.3 Development Phases

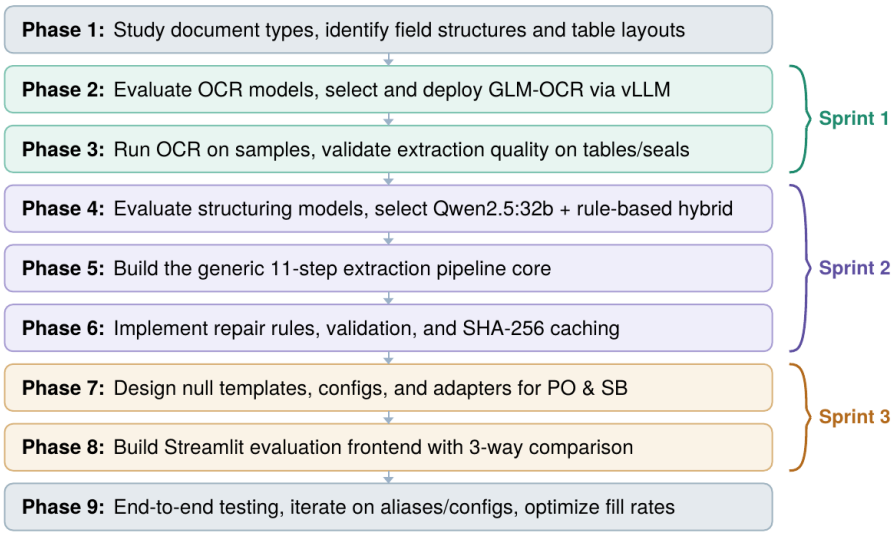


Figure 6.1: Development phases mapped to sprints

7 System Requirements

7.1 Hardware Requirements

Component	Minimum	Recommended
GPU	8 GB VRAM	24 GB+ VRAM (A100 / RTX 4090)
RAM	16 GB	32 GB+
Storage	50 GB free	100 GB+ (models + cache)
CPU	4 cores	8+ cores

Table 7.1: Hardware requirements

7.2 Software Requirements

Software	Version	Purpose
Python	3.12+	Core runtime for all components
UV	Latest	Fast package manager for GLM-OCR SDK
Ollama	Latest	Local LLM server hosting Qwen2.5
vLLM	Nightly	High-performance model serving for GLM-OCR
GLM-OCR	0.9B BF16	OCR model (∼1.8 GB download)
Qwen2.5:32b	32B params	AI extraction model (∼20 GB download)
PP-DocLayout-V3	Latest	Layout detection (25 label categories)
Streamlit	Latest	Evaluation frontend web framework
BeautifulSoup4	Latest	HTML table parsing in the pipeline
openpyxl	Latest	Color-coded Excel report generation
pandas	Latest	Data manipulation for evaluation

Table 7.2: Software and model requirements

8 Functional Requirements

ID	Requirement	Description
FR-01	PDF/Image Input	The system shall accept PDF files and image files (PNG, JPEG) as input documents.
FR-02	Layout Detection	The system shall detect and classify visual regions on each page — text blocks, tables, titles, formulas, seals, and images.
FR-03	OCR Extraction	The system shall extract text content from each detected region and produce per-page Markdown and JSON outputs.
FR-04	Result Merging	The system shall merge per-page OCR outputs into a single Markdown file and a single JSON file per document.
FR-05	Schema-Aligned Output	The system shall produce structured JSON output that matches the null template shape exactly — no extra keys, no missing keys.
FR-06	AI-Based Extraction	The system shall extract scalar field values (names, dates, addresses) using a local LLM with evidence-based prompting.
FR-07	Rule-Based Extraction	The system shall extract structured table data using deterministic column-index parsing from HTML tables.
FR-08	Value Repair	The system shall apply post-extraction fix rules (whitespace trimming, date normalization, currency stripping, etc.).
FR-09	Validation	The system shall validate extracted values for type correctness and required-field presence, producing a validation report.
FR-10	Response Caching	The system shall cache AI responses using SHA-256 keys to avoid redundant model calls on re-runs.
FR-11	Overlap Recall	The evaluation frontend shall compute per-page and overall field-level overlap recall for any prediction file.
FR-12	Alignment Report	The evaluation frontend shall generate downloadable, color-coded Excel files showing field-by-field GT vs Pred comparison.
FR-13	3-Way Comparison	The evaluation frontend shall support side-by-side comparison of Ground Truth vs MyModel vs GPT in a single Excel report.
FR-14	Plugin Registration	The system shall allow new document types to be added via config, adapter, and template files without modifying core pipeline code.

Table 8.1: Functional requirements

9 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Privacy	All processing must run locally. No document data shall be transmitted to any external server or cloud API at any stage of the pipeline.
NFR-02	Reproducibility	Given the same inputs and cache state, the system shall produce bit-identical outputs. SHA-256 disk caching ensures deterministic AI responses.
NFR-03	Extensibility	Adding a new document type shall require only 3 new files and 1 registry line. Zero modifications to any core pipeline file.
NFR-04	Accuracy	Rule-based table extraction shall be 100% deterministic. AI-based extraction shall achieve $\geq 85\%$ field-level match rate with repair rules enabled.
NFR-05	Usability	The evaluation frontend shall be operable by non-technical users with zero programming knowledge. All actions via upload, click, and download.
NFR-06	Maintainability	The codebase shall maintain strict separation between the generic core pipeline, document-specific plugins, and the evaluation layer.
NFR-07	Portability	The pipeline shall run on any Linux system with a CUDA-capable GPU. Model serving shall support Docker containerization.
NFR-08	Observability	Every pipeline run shall produce structured JSON logs with timestamps, field-level extraction status, and validation summaries.
NFR-09	Scalability	The system shall support horizontal scaling — processing N documents of the same type with a single schema and no per-document configuration.

Table 9.1: Non-functional requirements

10 Results

10.1 Shipping Bill Extraction

10.1.1 Our Final Pipeline

Status	Count	Percentage
MATCH	199	85.04%
MISMATCH	22	9.40%
MISSING_KEY	11	4.70%
NEAR_MATCH	2	0.85%
Grand Total	234	100.00%

Table 10.1: Shipping Bill — Our Pipeline results (234 fields)

10.1.2 GPT-Based Extraction

Status	Count	Percentage
MATCH	204	86.44%
MISSING_KEY	23	9.75%
MISMATCH	9	3.81%
Grand Total	236	100.00%

Table 10.2: Shipping Bill — GPT-based extraction results (236 fields)

10.2 Purchase Order Extraction

10.2.1 Our Final Pipeline

Status	Count	Percentage
MATCH	744	89.96%
MISMATCH	75	9.07%
MISSING_KEY	7	0.85%
NEAR_MATCH	1	0.12%
Grand Total	827	100.00%

Table 10.3: Purchase Order — Our Pipeline results (827 fields)

10.2.2 GPT-Based Extraction

Status	Count	Percentage
MATCH	527	63.57%
MISMATCH	269	32.45%
MISSING_KEY	33	3.98%
Grand Total	829	100.00%

Table 10.4: Purchase Order — GPT-based extraction results (829 fields)

10.3 Comparative Analysis

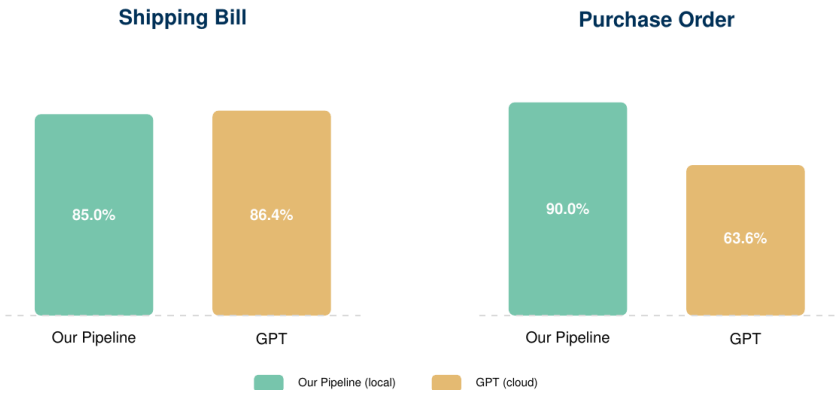


Figure 10.1: Match-rate comparison — Our Pipeline (fully local) vs GPT (cloud-based)

10.3.1 Key Findings

- On Purchase Orders, the local pipeline outperforms GPT by **26.4 percentage points** (90.0% vs 63.6%). The rule-based table parser achieves 100% accuracy on structured multi-page tables where GPT frequently misaligns columns.
- On Shipping Bills, both approaches perform comparably (∼85–86%), with GPT showing slightly fewer mismatches but more missing keys.
- The local pipeline has zero recurring cost and complete data privacy, while GPT requires per-token cloud API charges and document upload to external servers.
- `NEAR_MATCH` cases (0.1–0.9%) are typically formatting differences (e.g., date format variations) that could be resolved with additional repair rules.

11 Scalability Report

The pipeline is designed for horizontal scalability — once a schema is defined for a document type, it can process an unlimited number of documents of that type with zero additional configuration.

11.1 Horizontal Scaling — One Schema, N Documents

The core design principle: **define once, run everywhere**. A single schema (null template + config + adapter) serves as the blueprint for an entire document type. Whether you process 1 document or 10,000 documents, the setup is identical. Each document is processed independently with no shared state.

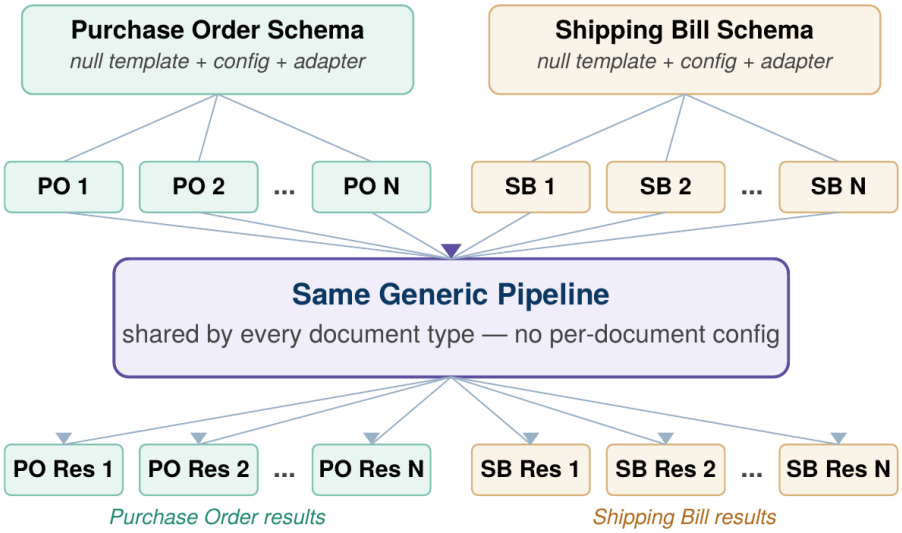


Figure 11.1: Horizontal scaling — one schema serves unlimited documents through the same pipeline

11.2 Key Characteristics Enabling Horizontal Scale

- **Stateless Processing** — Each document run is fully independent. No database, no shared memory, no inter-document dependencies. Documents can be processed concurrently across multiple machines or sequentially on a single machine.
- **No Per-Document Configuration** — All documents of the same type share a single schema, config, and adapter. Processing 1 document or 10,000 documents requires exactly the same one-time setup.
- **Disk-Backed Caching** — SHA-256 response caching means identical AI calls are never repeated. When processing a batch of similar documents, cache hit rates increase after the first few runs, effectively reducing compute cost to near-zero for repeated extraction patterns.

- **Batch-Ready** — A simple scripting loop over `run.py` processes an entire folder of documents. No orchestration framework, no job queue, no distributed system needed.
- **Multi-Machine Deployment** — The OCR server (vLLM) and AI server (Ollama) run as independent services on separate ports. They can be deployed on different machines, enabling GPU resource pooling across teams without any pipeline code changes.
- **Docker-Ready** — vLLM supports containerized deployment, enabling orchestration via Kubernetes or Docker Compose for enterprise-scale batch processing.

12 Advantages of the Project

1. **Complete Data Privacy** — All processing happens on local hardware. No document ever leaves the machine. This is critical for confidential commercial documents containing pricing, supplier terms, and trade secrets. Fully compliant with enterprise data governance policies.
2. **Zero Recurring Cost** — No API fees, no per-page charges, no subscription costs. Once the models are downloaded ($\sim$22 GB total), the pipeline runs for free indefinitely.
3. **Deterministic Table Extraction** — Rule-based parsing of structured tables is 100% accurate and reproducible. Unlike AI-based approaches that can hallucinate or misalign columns, column-index parsing always returns the correct value.
4. **Outperforms GPT on Complex Tables** — On Purchase Orders with multi-page table layouts, the local pipeline achieves 90% match rate vs GPT's 63.6% — a 26-point improvement. This demonstrates that domain-specific rules beat general-purpose AI for structured data.
5. **User-Friendly Evaluation** — The Streamlit frontend requires zero programming knowledge. Users upload three files, click buttons, and download color-coded Excel reports. The entire evaluation workflow is visual and intuitive.
6. **Plugin Architecture** — Adding a new document type requires zero changes to any core file. Three new files (template, config, adapter) and one registry line. The pipeline is future-proof and maintainable by any developer.
7. **Intelligent Caching** — SHA-256 disk-backed response cache means identical extractions are never repeated. Re-runs for debugging, testing, and iteration are near-instant ($<$5 seconds).
8. **Comprehensive Quality Reports** — The color-coded Excel reports with MATCH (green), NEAR_MATCH (orange), MISMATCH (yellow), and MISSING_KEY (red) make it immediately obvious where the pipeline succeeds and where it needs improvement — no manual inspection required.

13 Conclusion

This project demonstrates that a fully local, open-source pipeline can **match or exceed cloud-based AI services** for structured data extraction from commercial PDF documents — while offering complete data privacy and zero recurring cost.

The key insight is the **hybrid extraction strategy**: using AI (`Qwen2.5:32b`) for free-text fields where natural language understanding is needed, and rule-based column-index parsing for structured tables where deterministic accuracy matters. This combination achieves 85–90% overall match rates while maintaining 100% accuracy on table sections. On complex purchase orders, the local pipeline outperforms GPT by 26 percentage points.

The **plugin-based architecture** ensures the system scales to new document types without touching any core pipeline code — adding a new type is a matter of hours of manual document study, not weeks of model training or data annotation.

The **user-friendly Streamlit evaluation frontend** closes the feedback loop, allowing non-technical users to visually assess extraction quality through color-coded Excel reports with field-level MATCH, MISMATCH, NEAR_MATCH, and MISSING_KEY status tracking.

The pipeline runs entirely on local hardware with no API keys, no cloud dependency, and zero recurring cost, making it suitable for processing confidential commercial documents at scale within enterprise environments at Bosch Global Software Technologies.

BOSCH

Bosch Global Software Technologies
ERD Intern 2026